# Calculation of Vector Norms and Lengths

Meenu S

January 15, 2026

## 1 Objective

The objective of this program is to compute the Euclidean length of a vector(norm) in a real vector space $\mathbb{R}^n$ and to visualize the vector geometrically for two- and three-dimensional cases.

## 2 Mathematical Background

Let

$$\mathbf{v} = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n.$$

The Euclidean norm (or magnitude) of $\mathbf{v}$ is defined as

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}.$$

This quantity represents the distance of the vector from the origin.

## 3 Python Code

Listing 1: Python program to compute and visualize the Euclidean norm (length) of a vector

```
import numpy as np
import matplotlib.pyplot as plt
import traceback

def format_vector_for_display(v):
    """Formats a numpy array for display with commas, e.g.,
       [1.0000, 2.0000]."""
    return '[' + ', '.join(f'{x:.4f}' for x in v) + ']'

def get_dimension():
    """Gets a valid dimension from the user."""
    while True:
        try:
```

```
            dim_str = input("Enter the dimension of the vector
               space (e.g., 2 or 3): ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer for
               the dimension.")

def get_vector(dim):
    """Gets a valid vector from the user for a given dimension."""
    while True:
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements on one line, separated
           by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str = vector_str_input.split(',')

        if len(vector_str) != dim:
            print(f"Error: Incorrect number of elements. Please
               enter exactly {dim} elements.")
            continue

        try:
            v = np.array([float(x.strip()) for x in vector_str])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numbers.")

def calculate_and_plot_vector_length():
    """Calculates the length of a vector and plots it."""
    try:
        dim = get_dimension()
        v = get_vector(dim)

        length = np.linalg.norm(v)

        print(f"\nThe vector is: {format_vector_for_display(v)}")
        print(f"The length (magnitude) of the vector is:
           {length:.4f}")

        if dim == 2:
            fig, ax = plt.subplots(figsize=(8, 8))
            ax.set_title("2D Vector Plot")
```

```
        ax.quiver(0, 0, v[0], v[1], angles='xy',
            scale_units='xy',
                    scale=1, color='r',
                    label=f'Vector v =
                        {format_vector_for_display(v)}')
        ax.text(v[0] * 0.5, v[1] * 0.5,
                f'Length = {length:.2f}', fontsize=12,
                    color='blue')

        ax.plot([v[0], v[0]], [0, v[1]], 'k--')
        ax.plot([0, v[0]], [v[1], v[1]], 'k--')

        max_val = np.max(np.abs(v)) * 1.5 if np.max(np.abs(v))
            > 0 else 1.0
        ax.set_xlim([-max_val, max_val])
        ax.set_ylim([-max_val, max_val])
        ax.set_xlabel("X-axis")
        ax.set_ylabel("Y-axis")
        ax.axhline(0, color='grey', lw=0.5)
        ax.axvline(0, color='grey', lw=0.5)
        ax.grid(True)
        ax.set_aspect('equal', adjustable='box')
        ax.legend()

        output_filename = 'vector_length_plot_2d.png'
        plt.savefig(output_filename)
        print(f"\n2D plot saved as '{output_filename}'")

    elif dim == 3:
        fig = plt.figure(figsize=(12, 12))
        fig.suptitle("3D Vector and Orthographic Projections",
            fontsize=16)

        ax_3d = fig.add_subplot(2, 2, 1, projection='3d')
        ax_3d.set_title("3D Perspective View")
        ax_3d.quiver(0, 0, 0, v[0], v[1], v[2], color='r',
                     label=f'v =
                        {format_vector_for_display(v)}\nLength
                        = {length:.2f}')

        max_val = np.max(np.abs(v)) * 1.5 if np.max(np.abs(v))
            > 0 else 1.0
        ax_3d.set_xlim([-max_val, max_val])
        ax_3d.set_ylim([-max_val, max_val])
        ax_3d.set_zlim([-max_val, max_val])
        ax_3d.set_xlabel("X-axis")
        ax_3d.set_ylabel("Y-axis")
```

```
            ax_3d.set_zlabel("Z-axis")
            ax_3d.legend()

            ax_xy = fig.add_subplot(2, 2, 2)
            ax_xy.set_title("X-Y Plane (Top View)")
            ax_xy.quiver(0, 0, v[0], v[1], angles='xy',
                         scale_units='xy', scale=1, color='b')
            ax_xy.plot([v[0], v[0]], [0, v[1]], 'k--')
            ax_xy.plot([0, v[0]], [v[1], v[1]], 'k--')
            ax_xy.set_aspect('equal', adjustable='box')
            ax_xy.grid(True)

            ax_yz = fig.add_subplot(2, 2, 3)
            ax_yz.set_title("Y-Z Plane (Side View)")
            ax_yz.quiver(0, 0, v[1], v[2], angles='xy',
                         scale_units='xy', scale=1, color='g')
            ax_yz.plot([v[1], v[1]], [0, v[2]], 'k--')
            ax_yz.plot([0, v[1]], [v[2], v[2]], 'k--')
            ax_yz.set_aspect('equal', adjustable='box')
            ax_yz.grid(True)

            ax_xz = fig.add_subplot(2, 2, 4)
            ax_xz.set_title("X-Z Plane (Front View)")
            ax_xz.quiver(0, 0, v[0], v[2], angles='xy',
                         scale_units='xy', scale=1, color='purple')
            ax_xz.plot([v[0], v[0]], [0, v[2]], 'k--')
            ax_xz.plot([0, v[0]], [v[2], v[2]], 'k--')
            ax_xz.set_aspect('equal', adjustable='box')
            ax_xz.grid(True)

            plt.tight_layout(rect=[0, 0.03, 1, 0.95])
            output_filename = 'vector_plot_3d_views.png'
            plt.savefig(output_filename)
            print(f"\nMulti-view 3D plot saved as
               '{output_filename}'")

        else:
            print("\nPlotting is only supported for 2D and 3D
               vectors.")

    except KeyboardInterrupt:
        print("\nProgram interrupted by user.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        traceback.print_exc()

if __name__ == "__main__":
```

```
    calculate_and_plot_vector_length()
```

# OUTPUT

Enter the dimension of the vector space (e.g., 2 or 3): 3
Enter the 3 elements on one line, separated by commas (e.g., '1,2,3'): 2,0,6
The vector is: [2.0000, 0.0000, 6.0000]
The length (magnitude) of the vector is: 6.3246

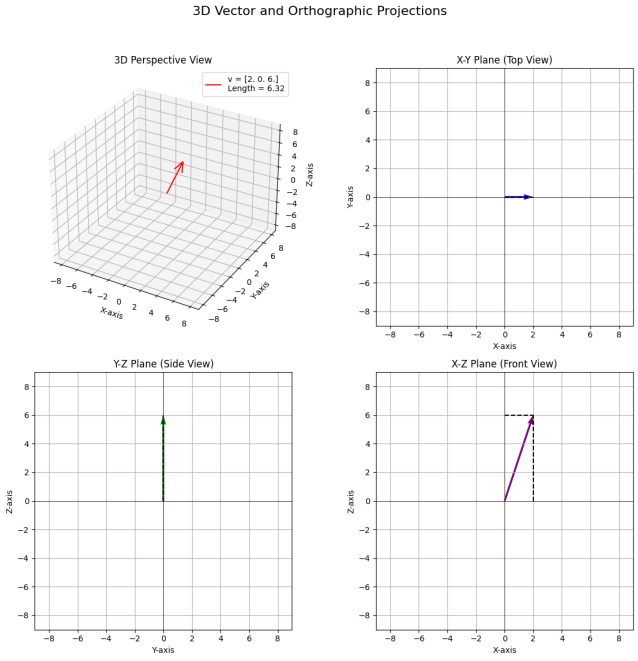


Figure 1: 3D vector showing its magnitude/length

Listing 2: Python program for computing norms of vectors and norm of their sum

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # Necessary for 3D plotting

def format_vector_for_display(v):
    """Formats a numpy array for display with commas to four
       decimal places."""
    return '[' + ', '.join(f'{x:.4f}' for x in v) + ']'

def get_dimension():
    """
    Asks the user to input the dimension of the vector space.
```

```
    """
    while True:
        try:
            dim = int(input("Enter the dimension of the vector
               space: "))
            if dim > 0:
                return dim
            else:
                print("Dimension must be positive.")
        except ValueError:
            print("Invalid input.")

def get_num_vectors():
    """
    Asks the user for the number of vectors.
    """
    while True:
        try:
            num = int(input("Enter the number of vectors: "))
            if num > 0:
                return num
            else:
                print("Number must be positive.")
        except ValueError:
            print("Invalid input.")

def get_vector(dim, name):
    """
    Inputs a vector of given dimension.
    """
    while True:
        elements = input(f"Enter {dim} elements for vector {name},
           separated by commas: ").split(',')
        if len(elements) != dim:
            print("Incorrect number of elements.")
            continue
        try:
            return np.array([float(x.strip()) for x in elements])
        except ValueError:
            print("Invalid input.")

def plot_vectors_2d(vectors, sum_vector):
    plt.figure()
    plt.grid()
    origin = np.zeros(2)

    for i, v in enumerate(vectors):
```

```
        plt.quiver(*origin, *v, scale=1, scale_units='xy',
            angles='xy', label=f'v{i+1}')

    plt.quiver(*origin, *sum_vector, scale=1, scale_units='xy',
        angles='xy', color='black', label='Sum')
    plt.legend()
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.title("2D Vector Sum")
    plt.gca().set_aspect('equal')
    plt.savefig("vector_plot_2d.png")

def plot_vectors_3d(vectors, sum_vector):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    origin = np.zeros(3)

    for i, v in enumerate(vectors):
        ax.quiver(*origin, *v, label=f'v{i+1}')

    ax.quiver(*origin, *sum_vector, color='black', label='Sum')
    ax.set_xlabel("X-axis")
    ax.set_ylabel("Y-axis")
    ax.set_zlabel("Z-axis")
    ax.set_title("3D Vector Sum")
    ax.legend()
    plt.savefig("vector_plot_3d.png")

def main():
    dim = get_dimension()
    num_vectors = get_num_vectors()

    vectors = []
    for i in range(num_vectors):
        vectors.append(get_vector(dim, f"v{i+1}"))

    sum_vector = np.zeros(dim)

    for v in vectors:
        sum_vector += v

    for i, v in enumerate(vectors):
        print(f"Norm of v{i+1}: {np.linalg.norm(v):.4f}")

    print(f"Norm of sum: {np.linalg.norm(sum_vector):.4f}")

    if dim == 2:
```

```
        plot_vectors_2d(vectors, sum_vector)
    elif dim == 3:
        plot_vectors_3d(vectors, sum_vector)

if __name__ == "__main__":
    main()
```

# OUTPUT

Enter the dimension of the vector space (e.g., 2, 3, 4): 2
Enter how many vectors you want to input: 2 — Enter Vectors —
Enter the 2 elements for vector v1, separated by commas (e.g., '1,2'): -1,0.25
Enter the 2 elements for vector v2, separated by commas (e.g., '1,2'): 4,-0.125
— Individual Vector Norms —
Norm of vector v1 [-1.0000, 0.2500]: 1.0308
Norm of vector v2 [4.0000, -0.1250]: 4.0020
— Sum of Vectors —
The sum of the vectors is: [3.0000, 0.1250]
The norm of the sum of vectors is: 3.0026

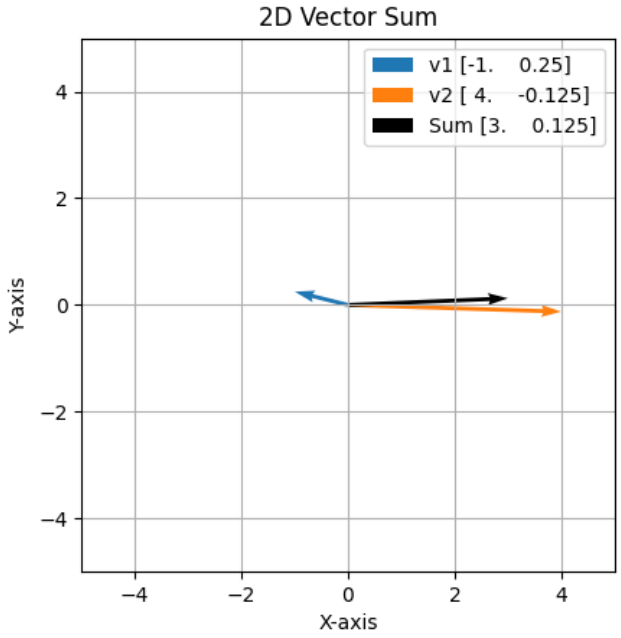


Figure 2: 2D vectors and their sums

## Distance between vectors

The distance between two vectors $u$ and $v$ in a vector space is defined as the norm of their difference:

$$\text{dist}(u, v) = \|u - v\|.$$

Here, $u - v$ represents the vector joining $v$ to $u$, and the norm $\|\cdot\|$ measures its magnitude. For the Euclidean norm in $\mathbb{R}^n$, this becomes

$$\|u - v\| = \sqrt{(u_1 - v_1)^2 + \cdots + (u_n - v_n)^2}.$$

This definition satisfies the properties of a distance: non-negativity, symmetry, and the triangle inequality.

Listing 3: Python code for finding the distance between 2 vectors

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Required for 3D plots

def get_dimension():
    """
    Asks the user for the dimension of the vector.
    Continues prompting until a valid positive integer is entered.
    """
    while True:
        try:
            dim_str = input("Enter the dimension of the vector: ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer.")

def get_vector(dim, vector_name="v"):
    """
    Gets a valid vector from the user for a given dimension.
    """
    while True:
        # Generate an example for the prompt based on the dimension
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements for {vector_name}
           vector , separated by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str_list = vector_str_input.split(',')

        if len(vector_str_list) != dim:
```

```
            print(f"Error: Incorrect number of elements. Please
                enter exactly {dim} elements.")
            continue

        try:
            # Convert string elements to floats and create a numpy
                array
            v = np.array([float(x.strip()) for x in
                vector_str_list])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
                numbers.")

def format_vector_for_display(vector):
    """
    Formats a numpy vector as a string with comma-separated
        elements.
    """
    return "(" + ", ".join([f"{x:.4f}" for x in vector]) + ")"

def calculate_vector_distance(vec1, vec2):
    """
    Calculates the Euclidean distance between two vectors.

    Args:
        vec1 (numpy.ndarray or list): The first vector.
        vec2 (numpy.ndarray or list): The second vector.

    Returns:
        float: The Euclidean distance between the two vectors.

    Raises:
        ValueError: If the dimensions of the vectors do not match.
    """
    # Convert inputs to numpy arrays to ensure consistent operations
    vec1 = np.array(vec1)
    vec2 = np.array(vec2)

    # Check if the dimensions (number of elements) of the vectors
        are the same
    if vec1.shape != vec2.shape:
        raise ValueError("Vectors must have the same dimension to
            calculate distance.")

    # Calculate the Euclidean distance
    # This is done by finding the difference between corresponding
```

```
        elements,
    # squaring each difference, summing them up, and then taking
        the square root.
    distance = np.linalg.norm(vec1 - vec2)

    return distance

def main_distance_calculator():
    """
    Main function to get two vectors from the user and calculate
        their Euclidean distance.
    """
    # Get the dimension from the user
    dim = get_dimension()

    # Get the first vector from the user
    vector1 = get_vector(dim, vector_name="first")
    print(f"\nFirst vector: {format_vector_for_display(vector1)}")

    # Get the second vector from the user
    vector2 = get_vector(dim, vector_name="second")
    print(f"Second vector: {format_vector_for_display(vector2)}")

    # Calculate the distance between the two vectors
    try:
        distance = calculate_vector_distance(vector1, vector2)
        print(f"\nEuclidean Distance between the two vectors:
            {distance:.4f}")

        # Plot the vectors if dimension is 2 or 3
        if dim == 2:
            plt.figure(figsize=(10, 10))
            ax = plt.gca()

            # Plot vector 1
            ax.quiver(0, 0, vector1[0], vector1[1], angles='xy',
                scale_units='xy', scale=1, color='blue',
                label=f'Vector 1:
                {format_vector_for_display(vector1)}', width=0.005,
                headwidth=5, headlength=8)
            # Plot vector 2
            ax.quiver(0, 0, vector2[0], vector2[1], angles='xy',
                scale_units='xy', scale=1, color='green',
                label=f'Vector 2:
                {format_vector_for_display(vector2)}', width=0.005,
                headwidth=5, headlength=8)
```

```
        # Plot line representing distance between vector heads
        ax.plot([vector1[0], vector2[0]], [vector1[1],
            vector2[1]], color='red', linestyle='--',
            label=f'Distance: {distance:.4f}', linewidth=2)

        # Calculate and plot the difference vector (vector2 -
            vector1)
        vector_diff = vector2 - vector1
        ax.quiver(0, 0, vector_diff[0], vector_diff[1],
            angles='xy', scale_units='xy', scale=1,
            color='purple', label=f'Difference (V2-V1):
            {format_vector_for_display(vector_diff)}',
            width=0.005, headwidth=5, headlength=8)

        # Set limits and labels
        max_abs = max(np.max(np.abs(vector1)),
            np.max(np.abs(vector2)),
            np.max(np.abs(vector_diff)), 1.5) # Ensure at least
            unit vectors are visible and includes difference
            vector
        ax.set_xlim([-max_abs - 0.5, max_abs + 0.5])
        ax.set_ylim([-max_abs - 0.5, max_abs + 0.5])
        ax.set_aspect('equal', adjustable='box')
        ax.axhline(0, color='gray', linewidth=0.5)
        ax.axvline(0, color='gray', linewidth=0.5)
        plt.xlabel('X-axis')
        plt.ylabel('Y-axis')
        plt.title(f'2D Vectors, Euclidean Distance, and
            Difference ({distance:.4f})')
        plt.legend()
        plt.grid(True)
        plt.show()
    elif dim == 3:
        fig = plt.figure(figsize=(12, 12))
        ax = fig.add_subplot(111, projection='3d')

        # Plot vector 1
        ax.quiver(0, 0, 0, vector1[0], vector1[1], vector1[2],
            color='blue', label=f'Vector 1:
            {format_vector_for_display(vector1)}',
            arrow_length_ratio=0.1)
        # Plot vector 2
        ax.quiver(0, 0, 0, vector2[0], vector2[1], vector2[2],
            color='green', label=f'Vector 2:
            {format_vector_for_display(vector2)}',
            arrow_length_ratio=0.1)
```

```
            # Plot line representing distance between vector heads
            ax.plot([vector1[0], vector2[0]], [vector1[1],
                vector2[1]], [vector1[2], vector2[2]], color='red',
                linestyle='--', label=f'Distance: {distance:.4f}',
                linewidth=2)

            # Calculate and plot the difference vector (vector2 -
                vector1)
            vector_diff = vector2 - vector1
            ax.quiver(0, 0, 0, vector_diff[0], vector_diff[1],
                vector_diff[2], color='purple', label=f'Difference
                (V2-V1): {format_vector_for_display(vector_diff)}',
                arrow_length_ratio=0.1)

            # Set limits and labels
            max_abs = max(np.max(np.abs(vector1)),
                np.max(np.abs(vector2)),
                np.max(np.abs(vector_diff)), 1.5) # Ensure at least
                unit vectors are visible and includes difference
                vector
            ax.set_xlim([-max_abs - 0.5, max_abs + 0.5])
            ax.set_ylim([-max_abs - 0.5, max_abs + 0.5])
            ax.set_zlim([-max_abs - 0.5, max_abs + 0.5])
            ax.set_xlabel('X-axis')
            ax.set_ylabel('Y-axis')
            ax.set_zlabel('Z-axis')
            plt.title(f'3D Vectors, Euclidean Distance, and
                Difference ({distance:.4f})')
            plt.legend()
            plt.grid(True)
            plt.show()
        else:
            print("\nPlotting is supported only for 2D and 3D
                vectors.")

    except ValueError as e:
        print(f"\nError: {e}")

if __name__ == "__main__":
    main_distance_calculator()
```

# OUTPUT

Enter the dimension of the vector: 3
Enter the 3 elements for first vector , separated by commas (e.g., '1,2,3'): 1,2,0

First vector: (1.0000, 2.0000, 0.0000)
Enter the 3 elements for second vector , separated by commas (e.g., '1,2,3'): -1,4,1
Second vector: (-1.0000, 4.0000, 1.0000)

Euclidean Distance between the two vectors: 3.0000

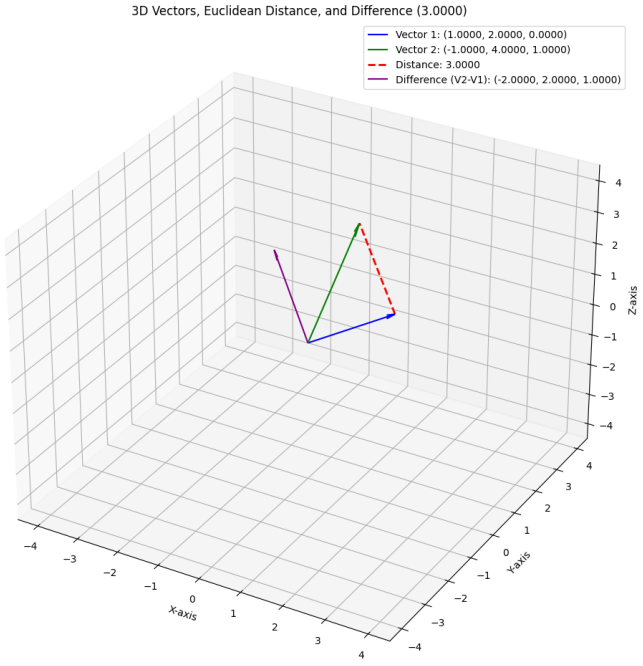


Figure 3: Distance between 2 3D vectors

# 4 Explanation of the Program

The first program requests the dimension of the vector space and validates the input. It then accepts a vector with the specified number of components and computes its Euclidean norm using NumPy's built-in linear algebra functions. In the second program, the norm of vectors and norm of their sum is calculated and in the third, using the idea of norm the distance between 2 vectors is calculated. Robust exception handling ensures safe execution and meaningful error messages.

# 5 Conclusion

In this work, vectors in finite-dimensional Euclidean spaces were studied using computational methods. Graphical representations in two and three dimensions further aided in visualizing vector addition and the geometric interpretation of norms. Thus, the program successfully demonstrates the application of linear algebra concepts using Python.